EECS 348 Group One

C++ Mathematical Compiler

User's Manual

Version 1.2

Revision History

Date	Version	Description	Author
06/May/26	1.1	Finished odd numbered sections	Lauren
07/May/26	1.2	Finished even sections and proofread	Collin

Table of Contents

1. Purpose

This user manual provides information on what the EECS 348 Mathematical Compiler does, how it installs, and how to use it. For information on the design and implementation of the Mathematical Compiler, please visit https://github.com/quinnsells/exprparser-Mathematical-Compiler.git to view the README, the source code in the 'src' folder, or the formal documentation in the 'documents' folder.

2. Introduction

2.1 Overview

The mathematical compiler allows the user to input a mathematical expression into the command line and then prints the answer. It also displays a detailed breakdown of the equations into tokens and shows the abstract syntax tree, which represents the order of operations. The compiler handles integers, decimals, and basic mathematical operations.

2.2 Installation

Installation of the mathematical compiler requires access to the GCC compiler and Git in a Linux environment. First, clone the repository by running the following command:

> git clone https://github.com/quinnsells/exprparser-Mathematical-Compiler.git

Next, ensure that the current directory is the main directory of the cloned repository. Then, compile the program by running the following command:

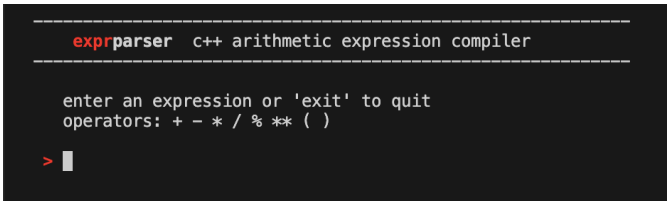
> make

Finally, run the program by running the following command:

> ./exprparser

3. Getting started

After running the program, you will be shown this prompt:



Here you may type the expression you would like to solve. The enter key will display its results.

3.1 Operator Usage

You may use the following (binary) operators:

Operation	Symbol	Format

Addition	+	[*Number*] + [*Number*]
Subtraction	-	[*Number*] - [*Number*]
Multiplication	*	[*Number*] * [*Number*]
Division	/	[*Number*] / [*Number*]
Modulo	%	[*Number*] % [*Number*]
Exponentiation	**	[*Number*] **[*Number*]

You may use the following (unary) operators to denote negative and positive numbers:

Operation	**Symbol**	**Format**
Positive	+	+ [*Number*]
Negative	-	- [*Number*]

You may also use parenthesis to control the order of operations. You must use the following rules when using parenthesis:

- For every open parenthesis, there must be one and only one closing parenthesis.
- You cannot use a closing parenthesis before the corresponding opening parenthesis.
- The opening parenthesis cannot occur directly before a binary operator [example: 5(*2)]
- The closing parenthesis cannot occur directly after a binary operator [example (5*)2)]
- Although valid mathematically, this compiler does not support multiplication via parenthesis [2(3)]

These operators adhere to the well-known PEMDAS and associativity rules. To clarify, here are the operations in the order they are solved:

1. Parenthesis
2. Unary Operators
3. Exponentiation
4. Multiplication, Division, and Modulo
5. Addition and Subtraction

When there are multiple operations occurring on the same level of precedence, the compiler calculates left to right for all operators except for exponentiation, which calculates right to left.

3.2 Number Usage

All rational numbers (see glossary of terms) are acceptable inputs to the Mathematical compiler. No special characters representing numbers (e.g. π) are acceptable. No decorative characters, such as commas, are acceptable. Numbers must have at least one integer preceding the decimal place. Finally, numbers can be too large. If this is the case, you may get an overflow error.

Valid examples:

- 3.2
- 0.003
- 5000
- +4

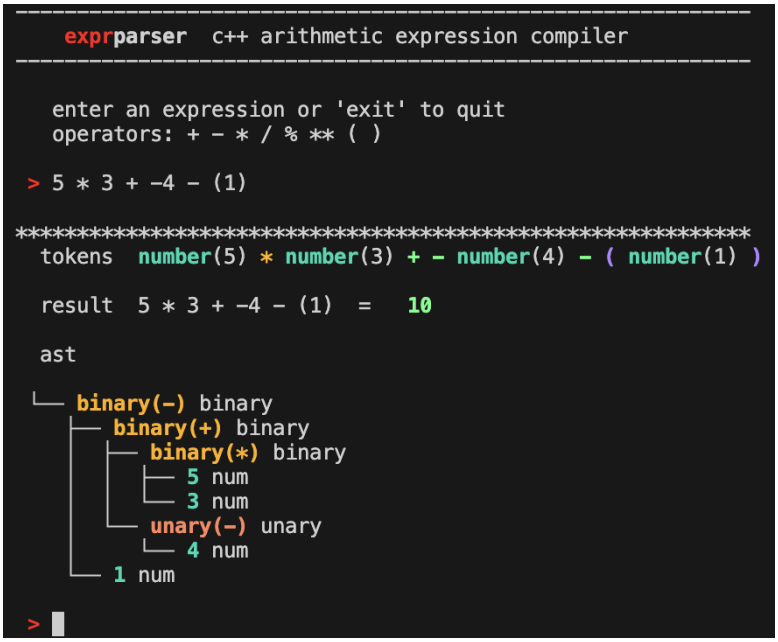
Invalid examples:

- 3i
- .003
- 5,000

- 3e2

3.3 Output

Here is an example output from a valid expression:



This is the input:

*> 5 * 3 + -4 - (1)*

It sees the number 5 multiplied to the number 3, and so on. This can assist in troubleshooting.

This line shows the input expression and result in green:

*result 5 * 3 + -4 - (1) = 10*

You may enter as many expressions as you like. From here, you can also exit the program. To close the program, type 'exit' and then enter.

4. Advanced features

This program displays advanced output in addition to the result. This information conveys how the compiler breaks down and ultimately calculates the expression.

4.1 Tokens

In addition to calculating the result, the program also displays how it calculates it. In the example screenshot provided in section 3.3, this line shows how the compiler interpreted the input:

*tokens number(5) * number(3) + - number(4) - (number(1))*

The compiler breaks the expression down into numbers, operators, and parenthesis. Each meaningful piece of data is represented as a token, which allows the compiler to handle multiple digit numbers, parenthesis, and unary operators.

4.2 Abstract Syntax Tree

The final piece of output provided by the program is the AST (Abstract Syntax Tree), noted by the *ast* label. This represents the order of operations that the compiler followed to calculate the result. The lowest levels of the tree are calculated first, and the results of those calculations are then used to calculate the next level, until the top of the tree is reached. In addition to the operator itself, the tree also displays the type of operator (binary or unary), which aligns with the number of inputs the operator needs.

5. Troubleshooting

Below are some common errors and solutions:

Error	**Solution**
error Unknown character '.' at position 0	This likely resulted from typing something like .345. Add a 0 in front to make 0.345. Refer to section 3.2.
error Unknown character ',' at position 1	This likely resulted from typing something like 5,000. Remove the comma.
error Expected 'end' but got '(' at position 1	This likely resulted from using parenthesis for multiplication such as in 2(5). Add a * in the middle: 2*(5).
error Cannot divide by zero	This resulted from an invalid mathematical equation that resulted in a division by zero.
error overflow - result too large to represent (exceeds 10^12)	The result is too large for this compiler to handle- use a different tool to solve this expression.

6. Examples

Below is a collection of screenshots demonstrating the program's use.

6.1 Basic Mathematical Operators

```
------------------------------------------------------------
    exprparser  c++ arithmetic expression compiler
------------------------------------------------------------

   enter an expression or 'exit' to quit
   operators: + - * / % ** ( )

 > 3+5

**************************************************************
  tokens  number(3) + number(5)

  result  3+5  =  8

  ast

 └── binary(+) binary
    ├── 3 num
    └── 5 num

 > 4*6

**************************************************************
  tokens  number(4) * number(6)

  result  4*6  =  24

  ast

 └── binary(*) binary
    ├── 4 num
    └── 6 num

 > 15-10

**************************************************************
  tokens  number(15) - number(10)

  result  15-10  =  5

  ast

 └── binary(-) binary
    ├── 15 num
    └── 10 num

 > 9/3

**************************************************************
  tokens  number(9) / number(3)

  result  9/3  =  3

  ast

 └── binary(/) binary
    ├── 9 num
    └── 3 num
```

```
> 8%3

************************************************************
 tokens  number(8) % number(3)

 result  8%3  =  2

 ast

└── binary(%) binary
    ├── 8 num
    └── 3 num

> 2**3

************************************************************
 tokens  number(2) ** number(3)

 result  2**3  =  8

 ast

└── binary(**) binary
    ├── 2 num
    └── 3 num
```

6.2 Advanced Expressions

```
> (3*2)+5-(3+4)**3

************************************************************
 tokens  ( number(3) * number(2) ) + number(5) - ( number(3) + number(4) ) ** number(3)

 result  (3*2)+5-(3+4)**3  =  -332

 ast

└── binary(-) binary
    ├── binary(+) binary
    │   ├── binary(*) binary
    │   │   ├── 3 num
    │   │   └── 2 num
    │   └── 5 num
    └── binary(**) binary
        ├── binary(+) binary
        │   ├── 3 num
        │   └── 4 num
        └── 3 num

> (4/2.3)**2%2

************************************************************
 tokens  ( number(4) / number(2.3) ) ** number(2) % number(2)

 result  (4/2.3)**2%2  =  1.02457467

 ast

└── binary(%) binary
    ├── binary(**) binary
    │   ├── binary(/) binary
    │   │   ├── 4 num
    │   │   └── 2.300000 num
    │   └── 2 num
    └── 2 num

> -(9/3)+(+9)**(2/3)

************************************************************
 tokens  - ( number(9) / number(3) ) + ( + number(9) ) ** ( number(2) / number(3) )

 result  -(9/3)+(+9)**(2/3)  =  1.32674871

 ast

└── binary(+) binary
    ├── unary(-) unary
    │   └── binary(/) binary
    │       ├── 9 num
    │       └── 3 num
    └── binary(**) binary
        ├── unary(+) unary
        │   └── 9 num
        └── binary(/) binary
            ├── 2 num
            └── 3 num
```

7. Glossary of terms

Term	Definition
Opening Parenthesis	The "(" symbol
Closing Parenthesis	The ")" symbol
Terminal	A text-based interface that allows the user to interact with a computer via commands.
MakeFile	This is a file that includes information for the computer on how to compile a program. The user does not need to view it, only use it via the term command "make" in the terminal
Git	Git is a commonly used version control system.
GitHub	This is a version control system which can be accessed on a browser. It contains everything you need to install and run the program.
Rational Numbers	A positive or negative number with a terminating decimal place. This does not include imaginary number or numbers with 'i' in them.
Integer	Any whole number, positive or negative.
Operator	A symbol that signifies a mathematical operation.
Unary Operator	An operator that takes one operand (e.g. positive or negative sign).
Binary Operator	An operator that takes two operators (e.g. +, -, /, *, %, **).
Operand	Something being operated on (in the context of this program, a number).
Modulo	Modulo is a binary operation which returns the remainder of the division between the two numbers. The sign of the result matches the sign of the first number in the division (dividend).
Abstract Syntax Tree	A data structure in computer science, its meaning is irrelevant to the usage of the mathematical compiler.
Compiler	Something that translates code to a more practical state. Its meaning is useless to the use of this program.
C++	The programming language this program was made with. Its meaning is irrelevant to the usage of this program.
Token	A singular piece of data (e.g. a number or an operator).
GCC	A C++ compiler that stands for GNU Compiler Collection.
Linux	An operating system based off the Linux kernel.

8. FAQ

Q. Does this program work on a non-Linux environment?

A. This program is written in C++, which works in every modern environment. However, the makefile is written using the GCC compiler, which is more complicated to install on a Windows or Mac machine.

Q. What's the difference between binary and unary operators?

A. Binary operators take two operands, and unary operators take one (for instance, positive or negative signs).

Q. I want to calculate an equation using a square root. How do I do that?

A. Square roots can be simulated by taking any number to the power of 1/2.

Q. I want to calculate an equation using a fraction. How do I do that?

A. Fractions can either be inputted through using parenthesis (*1/2*) or by taking a number by the power of –1.

Q. I want to calculate an equation using a logarithm/sine/other mathematical function. Can I do that?

A. Functions are not currently supported. They can be approximated, or another program can be used.

Q. I want to calculate an equation using π/e/another infinite constant. Can I do that?

A. There is no current support for characters that represent numbers. There are ways to approximate each number, and numbers can be manually approximated; however, exact numbers cannot be used.